Practice Set: C++ Keywords, Identifiers, and Variables

Section 1: Conceptual & Practical Questions (1–10)

Question 1: What are **Keywords** in C++? Explain how keywords affect compiler behavior during lexical analysis and state whether keywords allocate RAM by themselves.

Question 2: Compare the keyword capabilities of ANSI C with modern C++. Provide two examples of C++ specific keywords added for OOP, memory management, and data typing.

Question 3: Define an **Identifier** in C++. Explain the relationship between a user-defined identifier and physical memory addresses in RAM.

Question 4: List and explain the **5 Golden Rules** for defining valid identifiers in C++.

Question 5: State whether the following identifier names are **Valid** or **Invalid** according to C++ syntax rules:

a) 1stRank

b) student_age

c) totalAmount

d) UserAge

e) default

Question 6: Differentiate between **Declaration**, **Initialization**, and **Assignment** of a variable in C++, providing syntax examples for each.

Question 7: Explain modern C++ **Direct List Initialization** (e.g., int age{20};) introduced in C++11. How does it protect against implicit narrowing conversions compared to traditional copy initialization (e.g., int age = 3.99;)?

Question 8: Compare **Local Variables**, **Global Variables**, and **Static Local Variables** across two criteria: **Scope (Visibility)** and **Lifetime**.

Question 9: What happens when an uninitialized local variable (e.g., int score;) is read during program execution? Explain the concept of "garbage values" in memory.

Question 10: Write a C++ code snippet demonstrating a static local variable inside a function called trackCalls(). Show what output is produced when trackCalls() is executed three consecutive times.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): Why is capitalization critical when writing C++ keywords like if, int, or return?

Question 12 (Viva): What is the result during compilation if a developer tries to name a variable float or class?

Question 13 (Viva): Why is starting a user-defined variable name with a leading underscore (e.g., _MyVariable or __temp) considered bad practice in C++?

Question 14 (Viva): Are the identifiers userAge, UserAge, and USERAGE treated as identical or distinct by the compiler? Name the language rule behind this.

Question 15 (Viva): How many keywords were present in standard ANSI C compared to modern C++?

Question 16 (Viva): Where in memory is a global variable created and when is it destroyed?

Question 17 (Viva): What naming convention is recommended as best practice for C++ variables/functions versus C++ classes/structs?

Question 18 (Viva): Which C++ keyword replaces raw C memory allocation functions like malloc() and free() for dynamic memory handling?

Question 19 (Viva): How does the lexical tokenizer process source code like int age = 20; during compilation?

Question 20 (Viva): What is the lifetime of a static local variable defined inside a function block?

Answers & Explanations

Answer: 01

Keywords are pre-reserved words recognized directly by the C++ compiler with fixed, special meanings used to construct logic, control flow, and data types.

Keywords themselves do not allocate RAM storage. Instead, they instruct the compiler how to set up structural components, allocate variable memory, or execute instructions.

Answer: 02

Standard ANSI C features 32 keywords, whereas modern C++ includes over 90 keywords.

C++ Additions:

OOP: class, public, private, protected, virtual, this.

Memory/Utility: new, delete, constexpr, nullptr, namespace.

Data Types: bool, auto, wchar_t.

Answer: 03

An **Identifier** is a user-defined human-readable name given to program elements such as variables, functions, arrays, classes, and namespaces.

In RAM, physical memory is referenced by raw hexadecimal addresses (e.g., 0x7ffd9000). Identifiers act as convenient, symbolic labels pointing to those memory locations.

Answer: 04

Allowed Characters: Must start with an uppercase/lowercase letter (a-z, A-Z) or an underscore (_), followed by letters, digits, or underscores.

No Leading Digits: Cannot begin with a number (e.g., 1age is invalid).

No Whitespace: Spaces are strictly prohibited.

No Special Symbols: Symbols like @, #, %, $, & are forbidden.

Case Sensitivity: Uppercase and lowercase letters are distinct.

Answer: 05

a) 1stRank: Invalid (Cannot start with a digit).

b) student_age: Valid (Uses snake_case with valid characters).

c) total$Amount: Invalid (Contains forbidden special character $).

d) UserAge: Valid (PascalCase with valid letters).

e) default: Invalid (Matches a reserved C++ keyword).

Answer: 06

Declaration: Informs the compiler of the name and type without initializing a value (e.g., int age;).

Initialization: Assigns an initial value at the time of creation (e.g., int age = 20; or int age{20};).

Assignment: Overwrites an existing variable's memory value later in execution (e.g., age = 21;).

Answer: 07

Introduced in C++11, direct list initialization uses curly braces (int age{20};).

Protection: Traditional syntax like int x = 3.99; silently truncates data to 3 (copy initialization). List initialization (int x{3.99};) triggers a compilation warning or error to prevent accidental data loss.

Answer: 08

Variable Type	**Scope (Visibility)**	**Lifetime**
Local Variable	Inside defining block {}	Created at block entry, destroyed at block exit
Global Variable	Entire file / cross-file	Created at app start, destroyed at app exit
Static Local Variable	Inside defining block {}	Created once on first call, persists until app exit

Answer: 09

Reading an uninitialized local variable causes **undefined behavior**.

Declaring a variable without initialization leaves existing raw bit patterns in that allocated memory space, commonly known as **"garbage values"**.

Answer: 10

```
C++
#include <iostream>
void trackCalls() {
   static int callCount = 0;  ----->  Created once, retains state
   callCount++;
   std::cout << "Call count: " << callCount << std::endl;
}
int main()
{
   trackCalls(); // Output: Call count: 1
   trackCalls(); // Output: Call count: 2
   trackCalls(); // Output: Call count: 3
   return 0;
}
```

Answer: 11 Keywords in C++ must be written strictly in lowercase. Capitalizing them (e.g., If or Int) causes lexical tokenizer failure and compilation errors.

Answer: 12 Compilation will fail because reserved keywords match the compiler's internal reserved table and cannot be reused as user identifiers.

Answer: 13 Identifiers starting with underscores or containing double underscores are reserved by standard libraries for compiler internal implementations.

Answer: 14 They are treated as **three distinct identifiers** due to C++'s strict case sensitivity.

Answer: 15 Standard ANSI C features 32 keywords, whereas modern C++ includes over 90 keywords.

Answer: 16 A global variable is created at application startup and persists in memory until the application terminates

Answer: 17 camelCase is recommended for variables and functions (e.g., studentAge), while PascalCase is recommended for classes and structures (e.g., BankAccount).

Answer: 18 The new and delete keywords.

Answer: 19 The tokenizer splits int (matched against the reserved table as a data type keyword) and age (registered as a valid identifier in the symbol table)

Answer: 20 A static local variable is created once on the first function call and persists in memory until the application exits, retaining its value across calls.